

T fixtures – churn_rank.py RED-test corpus (plan.md T-D)

Revision: 1 Last modified: 2026-09-29T00:00:00Z

Contract: specs/004-fast-dev-cycles/plan.md §T-D05 ("Extra closure scrutiny for old, high-churn components") + specs/004-fast-dev-cycles/contracts/common-conventions.md (C-001..C-007, which churn_rank.py inherits like every Spec-004 tool). common-conventions.md 's own tool-map table lists \$FC/closure/churn_rank.py under "Plan tools that no contract in this directory covers (open gap — no contract is invented here ...)": **its interface, output and RED fixtures are fixed by the plan task text until a contract is written** — this file is that fixing, for THIS RED test's own GREEN-branch, following the house precedent set by fixtures/io_trace/README.md and fixtures/verdict_cache/README.md .

tasks.md T090 line, verbatim: "RED test constitution/scripts/fastcycle/tests/test_churn_rank_red.sh (ranking deterministic on a frozen history; control needle: ATM-277's files — 32 commits — rank in the top decile) (plan T-D05; SC-004)". plan.md T-D05's own "Protecting tests" line: "determinism of the ranking on a frozen history; control: a known high-churn path (ATM-277's files, 32 commits) ranks in the top decile."

The tool under test (not yet implemented – T-D /T , a later task)

constitution/scripts/fastcycle/closure/churn_rank.py , per plan.md's Project Structure (line 200:

```
churn_rank.py # component age/churn ranking for the risk order (T-D05) ). Today constitution/scripts/fastcycle/closure/ contains only .gitkeep (verified 2026-09-29: find closure -maxdepth 1 -name '*.py' |
```

`wc -l` → 0). T-D05's own "Work" line: "compute per-component age and churn from git; the top decile feeds the §11.4.132/§11.4.189 risk ordering: closures there additionally require the backstop run (T-C10) on the closing artifact."

Assumed CLI contract (UNCONFIRMED by any contracts/ file — DEFINED here for this RED test's own GREEN-branch, binding-if-adopted on T099's implementer):

```
churn_rank.py --as-of <YYYY-MM-DD> --components-file <path> \
              --repo-roots <path> --out <path> [--determinism-check]
```

- `--as-of <YYYY-MM-DD>` — **required** (C-003: "No tool reads the current time into the body; time-windowed tools take `--as-of <YYYY-MM-DD>` and require it"). Gates git history to commits with commit-date on or before `--as-of` in every named repo (so the ranking is a pure function of its explicit inputs, never of wall-clock "now" read internally).
- `--components-file <path>` — canonical JSON:

```
{  
  "components": [  
    {  
      "id": "<component-id>",  
      "paths": [{"repo": "<repo-alias>", "path": "<repo-relative-path>"}, ...]  
    }  
  ]  
}
```

A component MAY bundle paths that live in **different git repositories** (main monorepo, and any of its submodules — e.g. `presenter`, `constitution`) under distinct `repo` aliases, because a real-world "component" this project tracks (e.g. ATM-277's stale-frame handling) can legitimately span a submodule file and a parent-repo file.

- `--repo-roots <path>` — canonical JSON mapping each `repo` alias used above to an absolute git root: `{"<alias>": "<absolute path>", ...}`.
- `--out <path>` — canonical JSON result (C-002: sorted keys, `schema` field, `run_meta` excluded from `body_hash`).
- `--determinism-check` — per C-003's stated harness convention ("the harness `--determinism-check` flag present on every tool: runs twice internally, exits 1 with a diff on mismatch").

Per-path measurement: for each `{repo, path}` entry, `commits` = the count of `git -C <repo_root> log --follow --oneline --until='<as-of> 23:59:59' -- <path>` (rename-tracking via `--follow` is load-bearing — see Scenario 2 below); `first_commit_date` = the oldest commit date in that same query. A path that resolves to **zero** commits at or before `--as-of` (the path does not exist in the repo's history up to that date) is **not** silently reported as `commits: 0` — it is flagged `"missing_instrument": "path_not_found_in_repo:<alias>:<path>"` and its numeric fields are the literal `"UNMEASURED"` token (C-001/C-004: "no tool maps 'could not measure' to 0").

Per-component aggregation: `churn_commits` = sum of its resolvable paths' `commits` (a component with **all** paths missing is itself UNMEASURED, exit 1 — a finding, per C-001, never silently ranked with `churn_commits: 0`); `age_days` = `as_of` minus the OLDEST `first_commit_date` across its resolvable paths.

Ranking: components are sorted descending by `churn_commits` (primary), then by `age_days` descending (secondary tie-break — an older component among equal-churn ones is scrutinised first, consistent with T-D05's "old, high-churn components" framing). `rank` is the 1-indexed position in that sorted list.

`in_top_decile` = `rank <= ceil(N / 10)` where `N` is the total component count. This exact combination rule is **this RED test's own invented default** (T-D05's Work line does not pin one) — the implementer may refine it; the assertions below are built to hold under any reasonable choice that weights `churn_commits` heavily, per the note on Scenario 3.

The scenarios

Fixture	Scenario	Protecting-tests line it satisfies
<code>build_frozen_repo.sh</code> (built at test-run time into a tmpdir, not committed as a <code>.git</code> tree)	Golden-good / determinism	"ranking deterministic on a frozen history"
A synthetic throwaway repo built inline by the RED test	Rename-tracking negative control	§11.4.273 "the path is part of the instrument" — a native <code>git log --path</code> (no <code>--follow</code>) undercounts a renamed file
<code>golden_bad_missing_path.json</code>	Golden-bad	a component naming a path absent from the named repo's history must be flagged, never silently <code>churn_commits: 0</code>
the live, real-repository ATM-277 needle (built by the RED test itself, not a static file — see below)	Control needle (the task's literal ask)	"control needle: ATM-277's files — 32 commits — rank in the top decile"

`build_frozen_repo.sh` — golden-good / determinism

Deterministically constructs a small git repository from scratch: fixed author/ committer name+email, fixed `GIT_AUTHOR_DATE` / `GIT_COMMITTER_DATE` per commit (`2020-01-0N 12:00:00 +0000` , incrementing one day per commit — never "now"), fixed content, fixed commit messages. Two independent invocations of this script (into two separate tmpdirs) **MUST** produce **commit-SHA-identical** repositories — this is verified live by the RED test (never merely assumed) before any claim about `churn_rank.py` 's determinism is trusted, because a "frozen history" claim resting on a non-reproducible builder would prove nothing (§11.4.273). The repo has 5 files with exactly 1, 2, 3, 5 and 8 commits respectively

(`file_a.txt` through `file_e.txt`); the RED test confirms these exact counts via `git log --follow --oneline` on each independently-rebuilt copy, as a mechanism self-check, before ever invoking the (today, absent) tool against it.

Synthetic rename-tracking repo – negative control

A second small throwaway repo (built inline in the RED test, not a separate committed script — it exists only to prove one git-mechanics fact, not to be a reusable fixture): `old_name.txt` is created and committed twice, then `git mv` d to `new_name.txt` and committed twice more (4 commits total against one logical file). `git log --follow -- new_name.txt` MUST report all 4; `git log -- new_name.txt` (no `--follow`) MUST report only 2 — proving the exact false-negative failure mode a `churn_rank.py` implementation built WITHOUT rename-tracking would introduce (an old, frequently-renamed, genuinely high-churn component silently under-counted out of the top decile). This is a **negative control** in the §11.4.201(1) sense: it looks like it might legitimately be "2 commits" (what a naive `git log -- path` reports) but the correct answer is "4" — the component's true churn is not what its current path's naive history search returns.

`golden_bad_missing_path.json` – golden-bad

Names a component whose sole path is `constitution/scripts/fastcycle/tests/fixtures/churn_rank/THIS_PATH_DOES_NOT_EXIST_5f9c2a1b.txt` — a filename confirmed absent from the `constitution` submodule's history (control-needed: `git log --oneline -- <path>` returns zero rows). Once `churn_rank.py` exists, invoking it against this fixture MUST exit 1 (a finding, per C-001 — never 0, never a silent `churn_commits: 0`) and the component's `missing_instrument` field must literally name the path.

The real ATM-top-decile needle

Built by the RED test itself at run time, **never** a static frozen fixture, because it is inherently a claim about the project's own LIVE git history (both the `presenter` submodule and the parent monorepo), which keeps growing. The RED test:

1. Independently re-measures, live, via

```
git -C <presenter_root> log --follow --oneline -- Presenter/src/main/java/com/atmosphere/presenter/ VideoPlaybackDetector.kt and
git -C <repo_root> log --follow --oneline -- device/rockchip/rk3588/tests/test_display2_stale_frame_handoff.sh , the SAME two files
independently identified (§11.4.6, not guessed) as "ATM-277's files" from
docs/Fixed.md lines 5062/5188/5205 (the
VideoPlaybackDetector /"ATM-277 stale-frame class" and
test_display2_stale_frame_handoff.sh references) and confirmed via
docs/Issues.md:231 's ## ATM-277 heading + its Reopened-Details
evidence pointer.
```

2. **Captured FACT (measured 2026-09-29, git HEAD at that time):**

```
VideoPlaybackDetector.kt = 26 commits (presenter submodule, git log
--follow --oneline , oldest 2026-03-02); +
test_display2_stale_frame_handoff.sh = 6 commits (parent monorepo,
oldest 2026-06-04) = 32 commits combined — this EXACTLY matches the task
line's stated "32 commits" with no discrepancy to investigate or document
 (§11.4.6). The 6 parent-repo commits are the ones visible via git log --
follow --oneline -- device/rockchip/ rk3588/tests/
test_display2_stale_frame_handoff.sh from the monorepo root; one of
them ( 6a3dd85015a ) is itself titled "merge(ATM-690): ... + ATM-277 honest
§11.4.7 demotion + discovery sweep", corroborating that this file is genuinely
part of ATM-277's own fix/regression history, not an unrelated coincidence.
```

3. Builds a 10-component `--components-file` : the ATM-277 combined component (2 paths, spanning `repo: "main"` and `repo: "presenter"`) plus **9 other, real, independently-verified low-churn** components, each a single file inside the `constitution` submodule with exactly 1–3 commits
(`test_render_keys_red.sh` , `test_io_trace_red.sh` ,
`test_backstop_red.sh` , `test_precheck_slicer_red.sh` ,
`test_check_deps.sh` , `test_gate_order_red.sh` ,
`test_gate_shard_red.sh` , `test_gate_audit_red.sh` — each 1 commit;

`test_baseline_replay_red.sh` — 3 commits). Max among the 9 context components is 3; the ATM-277 component's combined churn (32, live-recomputed, $\geq$ documented) exceeds it by more than 10 $\times$, so under any reasonable churn-weighted ranking formula it lands at rank 1 of 10 — the top decile (`ceil(10/10) = 1` slot) — without the assertion being fragile to which exact combination rule the eventual T099 implementer chooses.

4. Once `churn_rank.py` exists: invokes it, asserts exit 0, asserts the `atm277_stale_frame` component's `rank == 1`, `in_top_decile == true`, and `churn_commits` equals the SAME live-recomputed sum from step 1 — never a hardcoded "32" as the pass/fail gate itself (a hardcoded number would silently go stale and produce a false FAIL the next time this file receives a legitimate new commit — ATM-277 is `Reopened`, still under active work). The "32" is captured evidence in this README and in this test's own echoed output, not the load-bearing numeric comparison.

What this RED test does NOT cover

- The `--determinism-check` flag's own internal double-run-and-diff behaviour is exercised only via the frozen-repo scenario; the golden-bad and real-ATM-277 scenarios invoke the tool once each (their own protecting-tests lines do not ask for determinism specifically).
- The exact combination formula for `age_days` feeding into the ranking beyond the simple tie-break stated above — T-D05's Work line leaves this open, and this RED test's assertions are deliberately built to hold regardless of the implementer's exact choice (see Scenario 3, step 3).
- The §11.4.132/§11.4.189 wiring of the top-decile output into the actual risk-ordered validation queue, and the T-C10 backstop-run requirement on closures of top-decile components — those are T099's own downstream wiring and a separate, later concern (T101/T102 territory), not this tool's own output-correctness contract.
- The paired §1.1 mutation of the real implementation — that is a mutation of code that does not yet exist (T093/T179 territory), not something a RED test against an absent tool can exercise.